

Informe técnico general (desarrollo)

Resumen

- **Proyecto:** DASHKPIS
Frontend: React + Vite + TS + Tailwind
Backend: Django + DRF
BD: SQL Server Azure / SQLite local
- **Objetivo cumplido:**
Implementar **formato configurable de reportes** en DashKPIs, permitiendo que el usuario exporte un PDF personalizado con:
 - * Mostrar/ocultar filtros
 - * Mostrar/ocultar resumen
 - * Mostrar/ocultar iniciativas
 - * Mostrar/ocultar secciones
 - * Tamaño de letra modificable
 - * Pie de página institucional
- **Correcciones clave:**
Se solucionaron problemas de integración FE/BE, serializer incompleto, PDF ignorando parámetros, y falta de renderizado de tipografía/pie de página.

Backend (Django + DRF)

Hechos clave

1. Serializer actualizado completamente

Se amplió ReportExportSerializer para aceptar las nuevas configuraciones:

- formato (Dict con toggles de visualización)
- mostrar_filtros, mostrar_resumen, mostrar_items, mostrar_secciones
- tamano_fuente
- pie_pagina

Esto permitió que las preferencias del usuario NO fueran descartadas por DRF.

2. Vista principal modificada (ReportExportPDFView)

Se integró toda la lógica para:

- Renderizar contenido condicionalmente según configuración del usuario.
- Aplicar tamaño de letra variable en el documento PDF.
- Insertar pie de página personalizado.

- Reordenar el flujo interno para que los elementos del PDF se rendericen en la secuencia correcta.

3. Motor PDF refactorizado

El método `_build_simple_pdf` se adaptó para:

- Cambiar el comando `/F1 XX Tf` dinámicamente (antes estaba fijo en 14).
- Agregar el pie de página dentro del contexto BT / ET.
- Mantener el espaciado vertical controlado (anteriormente irregular).

Checklist de errores relevantes resueltos (Backend)

- Serializer ignoraba parámetros de formato → agregado campo formato.
- PDF siempre usaba fuente 14 → ahora usa `tamano_fuente` enviado desde FE.
- Pie de página no aparecía → ajustado flujo BT/ET y agregado desplazamiento vertical.
- Filtros/resumen/items/secciones no eran opcionales → backend ahora respeta toggles.
- Payload incompleto → se alineó frontend y backend para transmitir datos correctamente.
- PDF no renderizaba correctamente cadenas largas → se reforzó `_wrap_text`.

Pendiente sugerido (Backend)

- Migrar motor PDF a ReportLab/WeasyPrint para soportar estilos avanzados.
- Agregar soporte de plantillas visuales (branding institucional).
- Internacionalización del PDF (ES/EN).

Frontend (React + Vite + TS)

Hechos clave

1. Nuevo panel: “Formato del reporte”

Se añadió un panel completo con:

- 4 toggles de visibilidad
- input numérico para tamaño de letra
- input texto para pie de página

Montado en la columna derecha del módulo **Reportes**.

2. Nuevo estado global `formatConfig`

```
const [formatConfig, setFormatConfig] = useState({  
  mostrarFiltros: true,  
  mostrarResumen: true,  
  mostrarItems: true,  
  mostrarSecciones: true,  
  tamanoFuente: 14,  
  piePagina: "DashKPI - Universidad Piloto de Colombia"  
});
```

3. exportPayload reestructurado

Ahora envía al backend:

```
formato: { ...formatConfig }
```

4. Botón Exportar PDF totalmente funcional

El frontend:

- Valida si hay datos
- Construye payload completo
- Llama al backend vía exportReportPdf
- Descarga el archivo con nombre dinámico

Checklist de errores relevantes resueltos (Frontend)

- PDF no reflejaba configuraciones → payload actualizado.
- Panel aparecía pero no tenía efecto → integración correcta con backend.
- Pie de página enviado pero ignorado → valor ahora viaja en formato.
- Tamaño de fuente no cambiado → ahora se envía y backend lo procesa.
- Filtros/resumen/items siempre estaban incluidos → control condicional implementado.

Pendiente sugerido (Frontend)

- Vista previa del PDF en tiempo real.
- Guardado de configuraciones preferidas por usuario.
- Opción de branding visual (logos + colores).

Base de datos (SQL Server)

Estado actual

No se realizaron cambios en BD para este feature.
Toda la lógica de personalización se maneja desde:

- Frontend: configuración del usuario
- Backend: construcción en memoria del PDF

Checklist de entorno y estabilidad

- Entorno de backend funcionando con Python 3.12
- Dependencias correctas
- CORS de Vite funcionando en puerto 5173

Pendiente sugerido (BD)

- Guardar configuraciones personalizadas de reporte por usuario (opcional futuro).

Operativa (cómo levantar)

Backend

activate venv

pip install -r requirements.txt

cp backend/.env.example backend/.env

python manage.py migrate

python manage.py runserver 8000

Frontend

npm ci

npm run dev

http://localhost:5173

Recomendaciones finales

- Documentar una guía del flujo de FE → BE → PDF.
- Migrar el PDF a un motor más avanzado.
- Agregar test unitarios y de integración para evitar regresiones en el PDF.
- Centralizar configuración de formatos del reporte para reusabilidad futura.